

승인된 기업만 지급할 수 있는가?

Track B · 지급 통제와 비공개 지급을 두 실습으로 검증

대상

은행.핀테크의
시니어 엔지니어

프로그램 1

Maroo 테스트넷
PCL · EAS · tOKRW

프로그램 2

로컬 Clairveil
deposit · transfer · scan

은행의 승인 상태를 체인 실행까지 전달합니다

기존 시스템의 심사와 온체인 지급 사이를 연결합니다

기존 시스템

기업 심사 · 이용 승인
지급 요청 생성



연결 지점

어떤 계정에 자격을 주고
어떤 경로로 지급할 것인가?



체인 실행

유효한 자격이면 지급
자격이 없거나 폐기되면 차단

익숙한 EVM 호출에서 체인 기능까지

Cosmos SDK → EVM → 프리컴파일의 역할을 연결합니다

Cosmos SDK

체인 기반과 모듈

EVM

컨트랙트 실행
ethers · JSON-RPC

프리컴파일

EVM에서 PCL·Privacy 등
체인 기능에 접근

EAS는 자격 기록, PCL은 실행 조건 검사

심사자 · 발급자 · 정책 관리자 · 지급 요청자를 구분합니다

EAS

발급자 · 대상 · 스키마
만료 · 폐기 · 데이터

PCL

지정 자격·조건을 평가
프록시 지급 허용 또는 거절

권한

실습용 프록시만 관리
실제 KYB·발급자 제한 미검증

지급 요청은 PCL에 등록된 프록시로 보냅니다

preCall → 지급 로직 → postCall

요청

기업 지급 계정
pay(recipient), value



정책 검사

전역 정책 + 프록시 정책
조건 위반 시 중단



결과

PolicyCheckPassed
영수증 · 수신자 잔액

실제 테스트넷 | 개별 거래의 처리 결과를 확인

70분 동안 두 환경의 증거를 모읍니다

설명 15분 → Maroo 25분 → Privacy 30분

00-15

업무 문제 · 구조
역할과 실행 경계

15-40

프록시 · EAS 실행
false/true 결과 해석

40-70

Privacy 설명·실습
공개 범위와 결과 판독

실습 준비 / 네트워크와 지급 계정을 확인합니다

chain ID 450815 · 테스트 전용 키 · 서로 다른 수신자

확인

```
npm run check:rpc  
npm run balance
```

서명 입력

로컬 .env에만 보관
키 화면 공유 금지

실행 규칙

--broadcast 명시
타임아웃 시 hash 먼저 확인

실제 테스트넷 | 개별 거래의 처리 결과를 확인

실습 1 / 프록시에서 지급과 차단을 비교합니다

```
npm run lab:pcl -- --broadcast
```

배포·등록

구현체와 전용 프록시
관리자 주소 확인

정상 지급

빈 Denylist → 0.001 지급
status 1 · 이벤트

차단 지급

송신자 차단 → 동일 요청
status 0 · 지급 변화 0

실제 테스트넷 | 개별 거래의 처리 결과를 확인

EAS는 발급 이후 인덱서 등록도 확인합니다

증명의 존재와 PCL이 찾을 수 있는 상태를 구분합니다

등록

실습 전용 schema
resolver 없는 데모

발급

자격 대상 · 만료 · 데이터
getAttestation으로 확인

인덱싱

indexAttestation(uid)
indexed=true 확인

실제 테스트넷 | 개별 거래의 처리 결과를 확인

실습 2 / 자격이 없으면 지급되지 않습니다

```
npm run lab:eas -- --broadcast
```

고정할 조건

같은 계정 · 수신자
0.001 tOKRW · 같은 함수

자격 없음

EasNoAttestationReceived
포함 영수증 status 0

상태 확인

수신자 증가 0
paymentCount 유지

실제 테스트넷 | 개별 거래의 처리 결과를 확인

실습 2 / 자격 발급 후 지급이 완료됩니다

유효한 데모 자격 + 인덱싱 → 정상 지급

자격 확인

schema · recipient 일치
미폐기 · 유효기간 내

실행

status 1
PolicyCheckPassed

지급 확인

수신자 +0.001 tOKRW
paymentCount +1

실제 테스트넷 | 개별 거래의 처리 결과를 확인

실습 2 / 자격을 폐기하면 다시 차단됩니다

승인 취소를 다음 지급 실행에 반영하는지 확인합니다

폐기

발급자가 revoke
revocationTime 확인

같은 지급

EasAttestationRevoked
status 0

해석

지급액 이동 없음
실패 거래 수수료는 발생

실제 테스트넷 | 개별 거래의 처리 결과를 확인

false인 증명도 지급이 통과했습니다

실제 데이터 값과 지급 결과를 대조한 실험

없음

자격 상태

지급 차단

false

자격 상태

0.001 tOKRW 지급 성공

true

자격 상태

0.001 tOKRW 지급 성공

실제 테스트넷 | 개별 거래의 처리 결과를 확인

유효한 자격과 데이터 값의 참·거짓은 다릅니다

이번 EAS_POLICY는 bool true를 지급 조건으로 강제하지 않았습니다

승인 표현

승인 대상에게만
해당 자격 증명 발급

취소 표현

false 발급에 의존하지 않음
기존 자격 폐기

추가 경계

누가 발급할 수 있는가?
운영 발급자 제한은 미검증

프로그램 2 / 지급 정보의 공개 범위를 다룹니다

PCL의 실행 통제와 Privacy의 정보 보호는 다른 질문입니다

PCL

이 지급을 실행해도 되는가?

Privacy

어떤 정보를
누구에게 공개하는가?

이번 범위

로컬 예치·지급·수신 확인
감사자 복호화 미검증

로컬 uclair | Maroo 호환성과 운영 보안은 별도 검증

Maroo와 Clairveil 결과를 분리합니다

외부 연동 대상과 독립 reference 구현

Maroo

실제 테스트넷 PCL·EAS
Privacy public interface 조회

Clairveil

고정 SHA · 로컬 체인
개발용 proof·uclair

미확보 연결

Maroo 호환 artifacts
state·witness·serialization

키와 공개 데이터의 경계를 먼저 확인합니다

비공개 note를 찾는 주체와 proof를 만드는 주체

공개 표면

거래 해시·공개 입력·이벤트
환경별 노출 확인

지갑·prover

키·note·witness 취급
원시 로그 공유 금지

감사 주체

별도 키와 접근 권한
설정과 복호화 검증은 다름

로컬 uclair | Maroo 호환성과 운영 보안은 별도 검증

실습 3 / 예치하고, 보내고, 수신 결과를 찾습니다

LOCAL_RUN.md · 준비된 artifacts · 새로운 run-dir

10

DEPOSIT

Alice 0 → 10 uclair



7

TRANSFER

준비 self batch 후
Bob에게 7 지급



3 + 7

SCAN

Alice 3 · Bob 7
반복 조회 결과 유지

로컬 uclair | Maroo 호환성과 운영 보안은 별도 검증

실습 3 / 성공은 받는 사람의 상태로 판단합니다

PUBLIC_RESULT.json을 기준으로 대조합니다

거래

deposit · self batch · transfer
포함 code = 0

잔액

Alice 10 → 3
Bob 0 → 7

재현

새 초기 상태 재실행
Bob 반복 scan 안정

로컬 uclair | Maroo 호환성과 운영 보안은 별도 검증

실패 결과도 개발자가 읽을 수 있어야 합니다

정책 오류 · 실행 실패 · 환경 문제를 구분합니다

Explorer

깨진 Decoded 대신
InDenylist(address)

자격 조회

발급 성공 ≠ 인덱싱 완료
count ≠ 유효성

환경·자료

RPC·artifact·키 오류
정책 거절로 세지 않음

지금 우리가 증명한 범위는 어디까지인가?

실행 결과와 업무 의미를 연결해 설명합니다

Maroo

정책이 실제 지급을
허용·차단했다

Local

비공개 note 지급 후
수취인이 결과를 확인했다

미검증

실제 KYB·발급자 제한
Maroo Privacy·감사 복호화

내 실행 증거를 남깁니다

성공 / 실패 / 미시도 / 기존 증거 분석

남길 것

환경·입력·해시·영수증
전후 상태·오류

보호할 것

개인키 · 원시 note
인증·개인정보

다시 볼 곳

README · 참가자 가이드
증거 색인 · 문제 해결